FeathersJS Group A

Minimum Viable Product Demo

4th March 2026

Agenda

01 MVP Scope Slide 03

02 Requirement/User Story Coverage Slide 05

03 MVP Functionality Slide 09

04 Demo Check In Slide 11

05 Reflection Slide 12

06 Next Steps Slide 14

Project Scope

FeathersJS integration with Large Language Models to facilitate ease of use

Live Project Creation

Build me a production-ready FeathersJS v6 real-time chat app in TypeScript with the following:

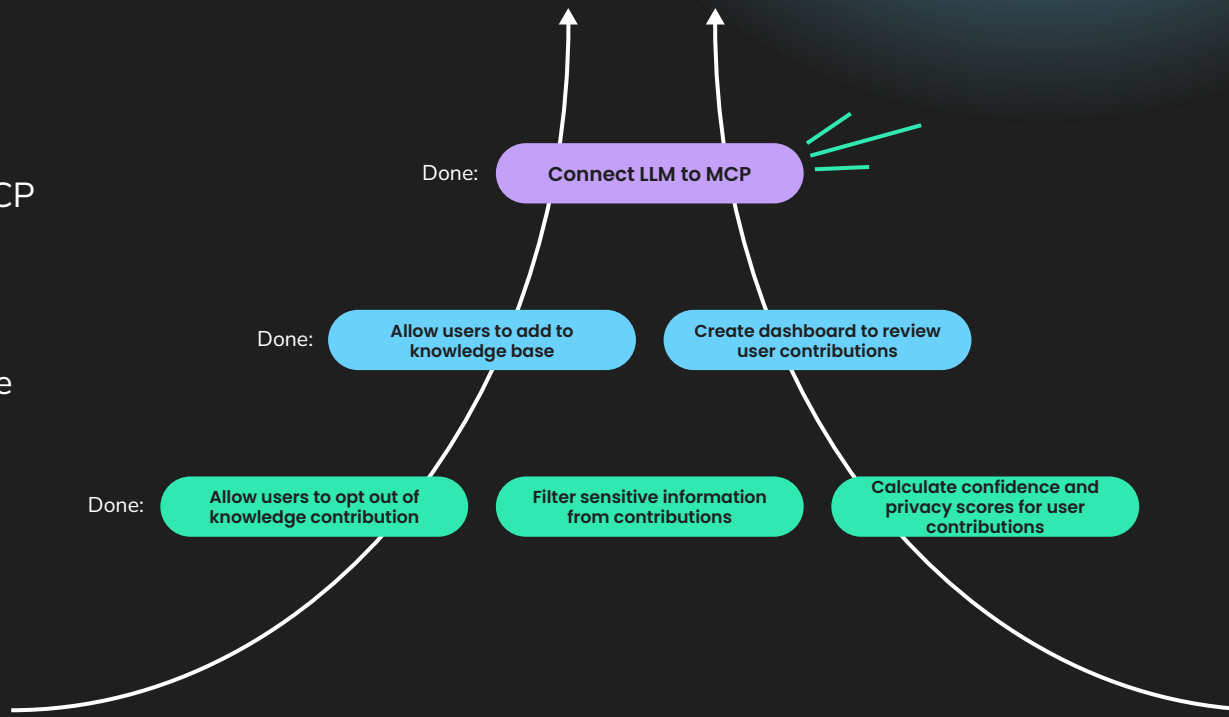
- SQLite via Knex adapter
- users and messages services with TypeBox schemas and resolvers
- Local auth (email/password) with proper v6 password hashing
- A hook that sets userId on new messages from the authenticated context
- Real-time channels so only authenticated users get message events
- A Socket.io client module that connects and authenticates

Give me complete, file-by-file code using only APIs that exist in FeathersJS v6.

Requirement Coverage

Top user story attributes:

- Use LLMs with knowledgeable MCP server to create feathers based projects
- Users have the ability to contribute to MCP knowledge base with opt in/out setting
- Admin dashboard to view and approve contributions

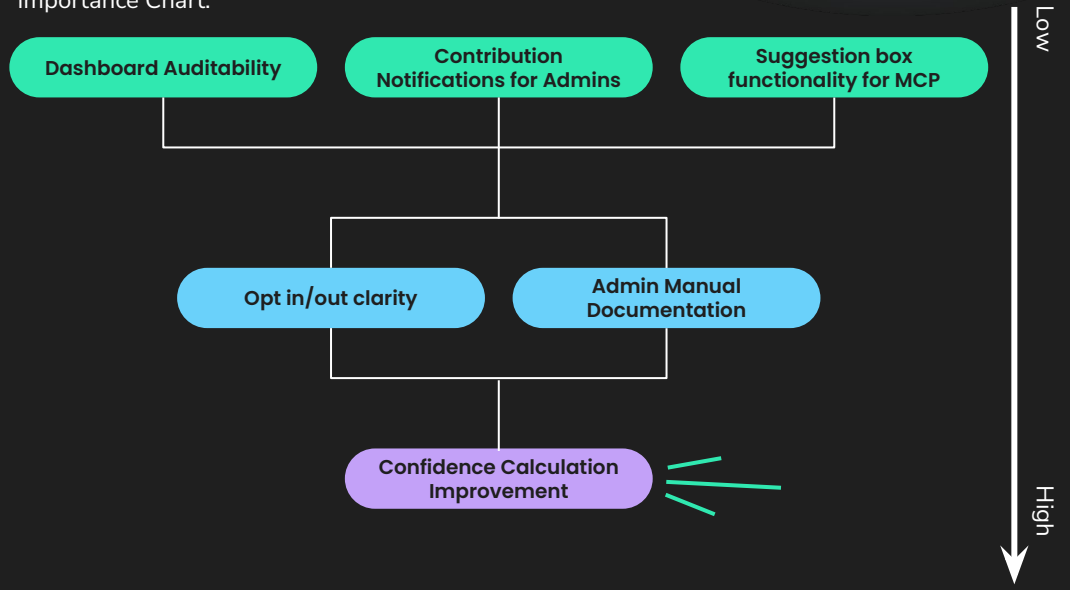


User Story Links

To be completed:

- Improved confidence and security scoring for user contributions
- Opt in/out clarity for users
- Admin dashboard and knowledge base auditability
- Improved CLI generation for feathers project creation
- Suggestion box style contributions to be made to server

Importance Chart:



Contribution Report and Explicit Consent

VsCode Agent Example (ChatGPT 5.0 mini)

Requirements

- System should provide popup to prompt user if they want to submit context

Server is running and `/tasks` responded with sample data. Ready to submit the implementation to the MCP KB — proceed with submission using `grantConsent: true` and `selfConfidence: 0.9` ? (yes/no)

User Stories

- (High) User who want to actively give back
- (High) Developer who does not want to contribute

Run Feathers Knowledge Submission – feathers-mcp (MCP Server)

POST-SOLUTION knowledge capture. Call only AFTER a Feathers solution has been implemented or verified and you can distill a reusable pattern Show More

Input

```
{
  "title": "Minimal Feathers v5 task manager
  (in-memory) with static frontend",
  "content": {
    "summary": "A minimal Feathers v5 application
    that provides an in-memory `tasks` service and a
    static frontend that uses the REST endpoints to
    list, create, patch, and remove tasks.",
    "prereqs": {
      "node": ">=14",
      "feathers": "5.x"
    },
    "steps": [
```

Note that MCP servers or malicious conversation content may attempt to misuse 'Code' through tools.

Allow Skip

Implemented Function

- Integrated LLM will always ask the user with a contribution report when writing data

Now I'll submit a sanitized knowledge-candidate summarizing the implemented Feathers task-tracker (includes code snippets and steps).

✓ Ran Feathers Knowledge Submission – feathers-mcp (MCP Server)

✓ Completed: *Submit Feathers context (knowledge candidate)* (5/5)

Status: Completed the minimal Feathers task-tracker and submitted the context.

- **Project files**: Created feather-mcp-test/package.json, feather-mcp-test/src/app.js, feather-mcp-test/public/index.html, and feather-mcp-test/README.md.
- **Run verified**: Installed deps and started the server; `/tasks` returned a seeded task and the static frontend is served at root. Server listening on http://localhost:3030.
- **Knowledge submitted**: Submitted a sanitized Feathers example as a knowledge candidate. Entry ID: 1e292e64-5cc1-4564-979b-aa26f79cb3a8.
- **Note about Feathers version**: @feathersjs/feathers v6 is not available on npm; I used v5-compatible packages so the app runs locally. Patterns are applicable to v6 docs where available.

Admin Dashboard and Auditing

Requirements

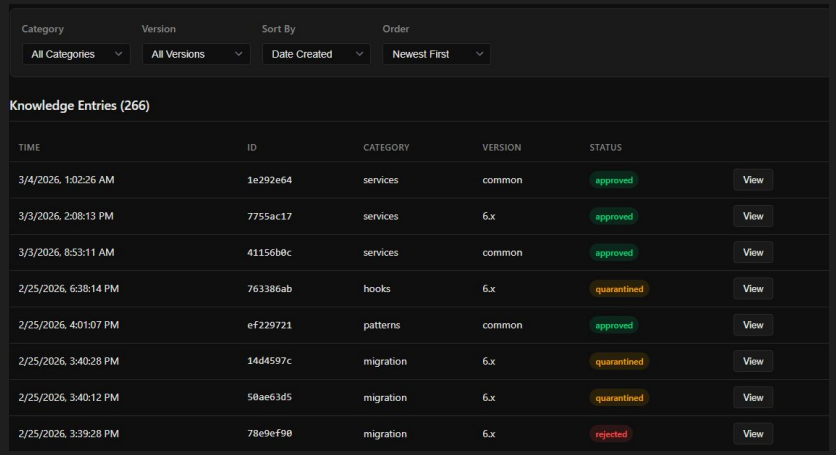
- The system should be able to be accessed by admin

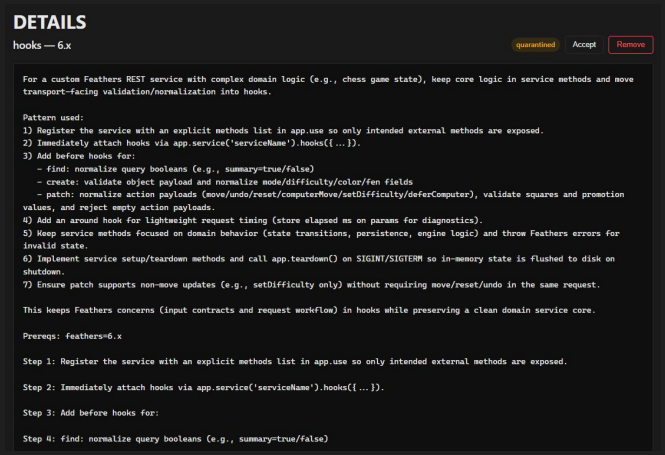
User Stories

- (High) Admin who need to view details on what is being submitted to the MCP server
- (Medium) For context to automatically update to the MCP server

Implemented Function

- Created a admin dashboard where all committed context can be viewed along with relevant information





Does it Work?

Reliability

Application was ~**90% functional** and had **1 issue** related to feathers v6 out of the box

Performance

Basic product was created within ~**20 minutes** including time taken to give permissions

Adherence

~**95%** of the code strictly followed v6 practices

Claude Code: Opus 4.6

Comparison: With *vs* Without MCP

Without MCP

- Fell back to v5 practices
- Version inconsistencies
- Slow learning speed
- Code structure deviated from v6 conventions

With MCP

- MCP guarded against version drift
- Compiled and ran without errors
- Fast information retrieval
- Code structure aligned with v6 conventions

1. Creating the prompt (ChatGPT 5.2 Thinking)
2. Creating the applications (Claude Code w/ Opus 4.6)
3. Comparing the applications against official v6 documentation (Gemini 3.0 Preview & Opus 4.6)
4. Cross Validation/Confirmation (Claude Code w/ Opus 4.6))

Demo Check In!

Reflection

Issues	LLM Submission Reliability	Confidence Scoring	MCP Tool Calls Consistency
Assumption	LLM-generated knowledge entries would generally be reliable enough to automatically add to the knowledge base	Use automated confidence scoring algorithm to determine if contribution should be added to knowledge base	LLMs would consistently rely on MCP tools such as retrieve and contribute
Observed	• Subtle inaccuracies • Hallucinated framework details	• Evaluating quality of generated summaries proved difficult • Accuracy, completeness, usefulness hard to quantify automatically	• Dependent on agent and model • Some IDE agents call more consistently than others
Risk	• Propagate errors for future users • Make knowledge base worse • Possible continuous cycle	• Easy scoring → bad contributions • Hard scoring → No contribution	• Unpredictable MCP tool calls • Inaccurate responses
Insight	• Validation layer still required	• Not purely algorithmic • Human review still necessary	• MCP tools also need designing to increase call consistency

Confidence Update

Current:

- LLM self confidence score
- Server side pattern recognition

Planned:

- Server side LLM using provided API key
- Server side pattern recognition

Next steps

Top Remaining Features

- Fully fleshed out confidence and security system
- Change MCP context directly on admin dashboard
- Developer send documentation suggestions to admin

Biggest Remaining Risk

- Unlimited user submission resulting in future project jeopardy

Blocker

- API costs for confidence scoring

Thank you!